



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3106 Application Development

SECTION 02

Final Test

Restaurant Waiter Order System

LECTURER: DR. SEAH CHOON SEN

Student Name	Matric No.
TEH RU QIAN	A23CS0191

1.0 Introduction

This report documents the development of a mobile application named 'resto_order', a waiter-facing order management system built for the Android platform. The application is designed to assist restaurant staff in managing menu items and customer orders efficiently through a simple and intuitive interface.

The app was developed using the Flutter framework with the Dart programming language and uses Google Firebase Firestore as its cloud-based backend database. It supports real-time data synchronisation, meaning any changes to menu items or orders are instantly reflected on the device without needing to manually refresh.

The system does not require any login or registration. Waiters can open the app and immediately begin managing the menu or taking orders. The application covers all core CRUD operations, which are Create, Read, Update and Delete, for both menu items and customer orders.

The main purpose of the system is to replace manual order-taking methods with a digital solution that reduces errors, tracks order status in real time and keeps an accurate history of all transactions even if menu prices change over time.

2.0 Software, Language, Tools, API, Templates used

Item	Details
Programming Language	Dart
Framework	Flutter
IDE	Visual Studio Code / Android Studio
Backend / Database	Google Firebase Firestore
Platform Tested	Android (Oppo device)
State Management	setState (built-in Flutter)
Package: firebase_core	Version 4.11.0
Package: cloud_firestore	Version 6.6.0
UI Components	Flutter Material 3 Widgets
Version Control	Git

3.0 Development Steps

Step 1 - Project Setup

A new Flutter project named 'resto_order' was created. The project was configured for Android deployment and the 'pubspec.yaml' file was updated to include the 'firebase_core' and 'cloud_firestore' dependencies.

Step 2 - Firebase Configuration

A Firebase project was created in the Firebase Console. The Android app was registered and the 'google-services.json' file was downloaded and placed in the android/app directory. Firebase was initialised inside the 'main.dart' file using WidgetsFlutterBinding.ensureInitialized() before the app launches.

Step 3 - Firestore Data Modelling

Three Firestore collections were designed to store the app data. The 'menu_items' collection stores each menu item with its name, price, category and availability. The 'orders' collection stores order information including table number, status, total amount and creation timestamp. The 'order_items' collection stores individual line items linked to each order with name and price snapshots to preserve accuracy for historical orders.

Step 4 - FirestoreService Class

A centralized FirestoreService class was created to handle all database operations. This class contains all methods for reading, writing, updating and deleting documents in Firestore. Screens in the app do not call Firestore directly and instead use this service class, which keeps the code organised and easy to maintain.

Step 5 - App Navigation Structure

The HomeScreen was set up with a BottomNavigationBar containing two tabs, one for the Menu and one for Orders. This gives waiters quick access to both main functions of the app from a single screen.

Step 6 - MenuListScreen Development

The menu listing screen was built using a 2-column GridView. Category filter chips at the top allow the waiter to filter items by Food, Drink, Dessert or Snack. Each card displays a colour-coded image placeholder based on category, the item name, price and an available or unavailable badge. Edit and delete icon buttons are also shown on each card. A LayoutBuilder was used inside each card to calculate the image height as 45 percent of the card height to prevent overflow on different screen sizes.

Step 7 - MenuFormScreen Development

A form screen was created for adding new menu items and editing existing ones. The form includes fields for item name, price, a chip-based category selector, and an availability toggle

switch. Before saving to Firestore, a confirmation AlertDialog is shown to the user. This screen is opened via the floating action button on the MenuListScreen.

Step 8 - OrdersListScreen Development

The orders listing screen was built using a ListView that displays orders sorted by creation time in descending order. A horizontal filter bar allows filtering by status, which includes All, Pending, Preparing, Served and Paid. Each order card shows the table number, time, total and a colour-coded status badge. The screen listens to a live Firestore stream so updates appear immediately.

Step 9 - NewOrderScreen Development

The new order screen allows a waiter to enter a table number and select items from a live 2-column menu grid. Each item card has plus and minus buttons to adjust quantity. The running total is displayed at the bottom. When the waiter taps Place Order, a confirmation dialog shows the table number, all selected items with quantities and the total. On confirmation, a Firestore batch write atomically creates the order document and all related 'order_item' documents at the same time.

Step 10 - OrderDetailScreen Development

The order detail screen shows the full details of a selected order, including a 4-step status stepper, a status banner, a scrollable list of order items with quantities and line totals, and a grand total. An action button advances the order through the statuses from Pending to Preparing to Served to Paid. Each status change requires confirmation via a dialog. A delete button is shown only when the order is in Pending status, and tapping it performs a batch delete of the order and all its associated 'order_items'.

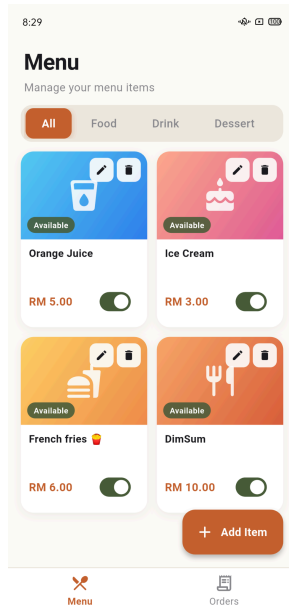
Step 11 - Navigation and Filter Synchronisation

After any action in OrderDetailScreen, the screen calls Navigator.pop with the next status value. OrdersListScreen receives this value and calls setState to update the active filter, so the waiter lands on the correct status tab automatically. The same logic applies after placing a new order, which returns the user to the Pending filter.

4.0 System Interface

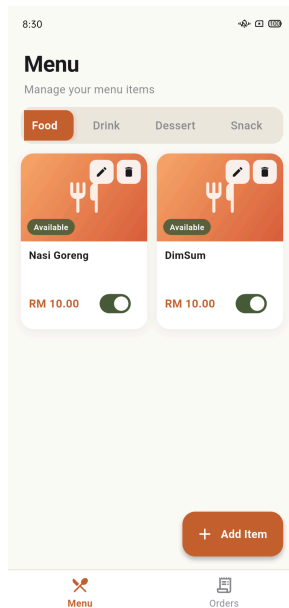
Menu List Screen

This screen shows the main menu management page with all items displayed in a 2-column grid. Category filter chips at the top allow the waiter to filter by Food, Drink, Dessert or Snack and each card displays the item name, price, availability badge and edit or delete buttons.



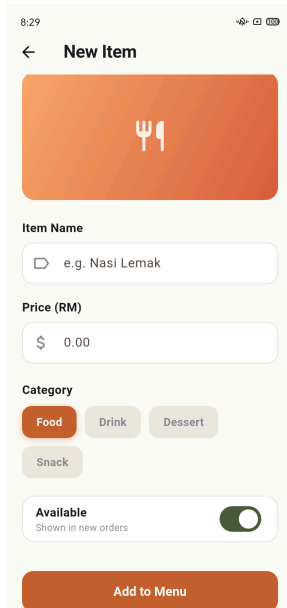
Menu List Screen (Food Category Filter)

This screen shows the menu list filtered to the Food category only, displaying Nasi Goreng and DimSum as the available food items. The active Food filter chip is highlighted in orange, confirming that the category filter is working correctly.



New Item Form

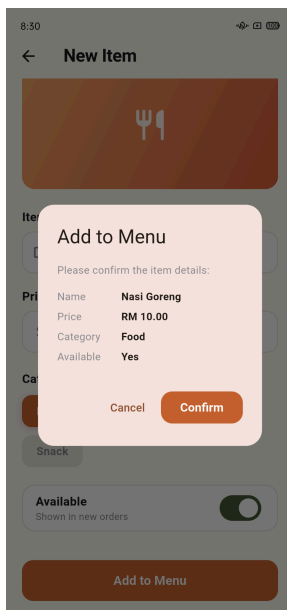
This screen shows the form for adding a new menu item, with input fields for item name and price, a chip selector for category and an availability toggle switch. The Add to Menu button at the bottom submits the form after a confirmation dialog is shown.



A screenshot of a mobile application's 'New Item' form. The form is set against a light cream background. At the top, there's a status bar showing 8:29 and a back arrow. Below the title 'New Item' is a large orange square with a white fork and knife icon. The form contains several input fields: 'Item Name' with a placeholder 'e.g. Nasi Lemak', 'Price (RM)' with a placeholder '\$ 0.00', and a 'Category' section with four chips: 'Food' (selected), 'Drink', 'Dessert', and 'Snack'. There's also an 'Available' toggle switch, which is currently turned on, with the text 'Shown in new orders' below it. At the bottom is a large orange button labeled 'Add to Menu'.

Add to Menu Confirmation Dialog

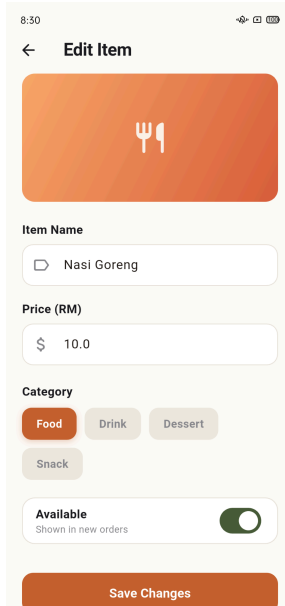
This screen shows the confirmation dialog that appears before a new menu item is saved, displaying the item name, price, category and availability for the waiter to review. The waiter can tap Confirm to save the item to Firestore or Cancel to go back and make changes.



A screenshot of the 'Add to Menu' confirmation dialog. The dialog is a light pink rectangle with rounded corners, centered over a dimmed version of the 'New Item' form. The dialog has a title 'Add to Menu' and a subtitle 'Please confirm the item details:'. It lists the item details: 'Name: Nasi Goreng', 'Price: RM 10.00', 'Category: Food', and 'Available: Yes'. At the bottom of the dialog are two buttons: 'Cancel' and 'Confirm'.

Edit Item Form

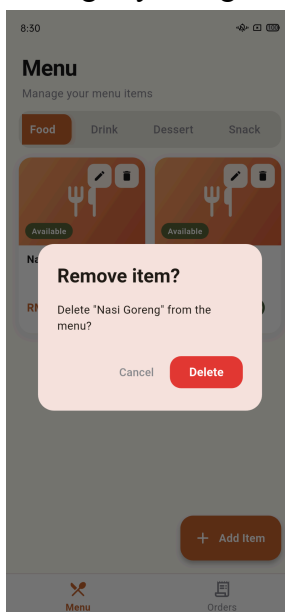
This screen shows the edit form with the existing item details pre-filled, including the name Nasi Goreng, price of RM 10.00, category set to Food and availability toggled on. The Save Changes button at the bottom updates the item in Firestore after the waiter confirms the changes.



The screenshot shows the 'Edit Item' screen. At the top, there's a back arrow and the title 'Edit Item'. Below this is a large orange square with a white fork and knife icon. The form contains the following fields: 'Item Name' with a text input containing 'Nasi Goreng'; 'Price (RM)' with a text input containing '\$ 10.0'; 'Category' with four buttons: 'Food' (selected), 'Drink', 'Dessert', and 'Snack'; and an 'Available' toggle switch which is turned on. At the bottom is a large orange button labeled 'Save Changes'.

Delete Item Confirmation Dialog

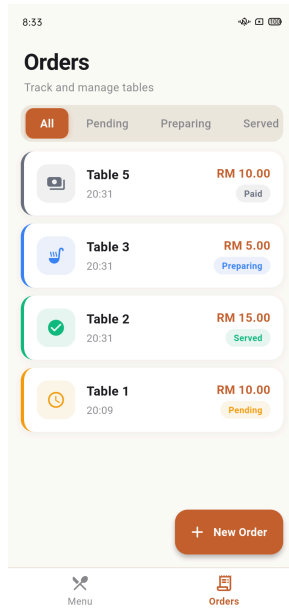
This screen shows the confirmation dialog that appears when the waiter taps the delete icon on a menu item card, asking whether to permanently remove Nasi Goreng from the menu. The waiter must tap the red Delete button to confirm the deletion or tap Cancel to dismiss the dialog without making any changes.



The screenshot shows the 'Menu' screen with a confirmation dialog overlay. The background is dimmed, showing the 'Menu' title, a subtitle 'Manage your menu items', category tabs ('Food', 'Drink', 'Dessert', 'Snack'), and menu items. The dialog is titled 'Remove item?' and contains the text 'Delete "Nasi Goreng" from the menu?'. It has two buttons: 'Cancel' and 'Delete' (in red). At the bottom of the screen, there is a '+ Add Item' button and a bottom navigation bar with 'Menu' and 'Orders' icons.

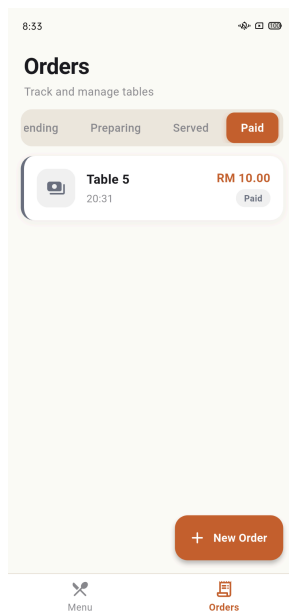
Orders List Screen (All Filter)

This screen shows the orders list with the All filter active, displaying four orders across different statuses including Paid, Preparing, Served and Pending, each with a colour-coded status badge and left border. The list is sorted by most recent first and updates in real time via a Firestore stream.



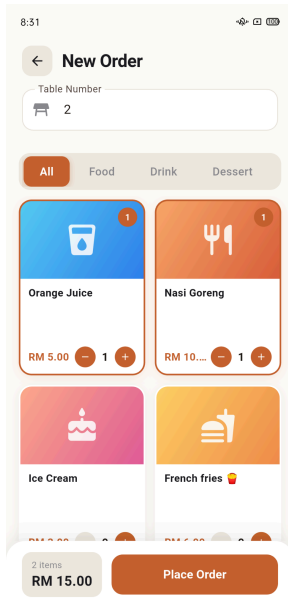
Orders List Screen (Paid Filter)

This screen shows the orders list filtered to the Paid status only, displaying Table 5 as the single completed and paid order. The Paid filter chip is highlighted, confirming that the status filter navigation works correctly after completing an order.



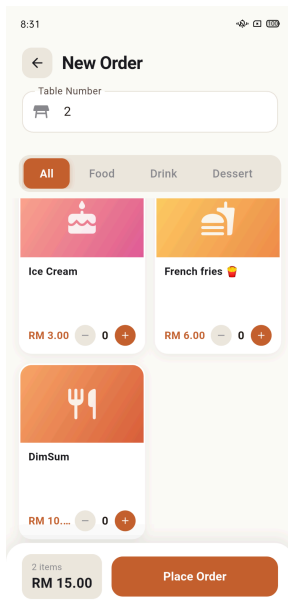
New Order Screen (Items Selected)

This screen shows the new order screen where the waiter has entered Table 2 and selected one Orange Juice and one Nasi Goreng using the plus and minus quantity buttons. The running total of RM 15.00 and item count are displayed at the bottom above the Place Order button.



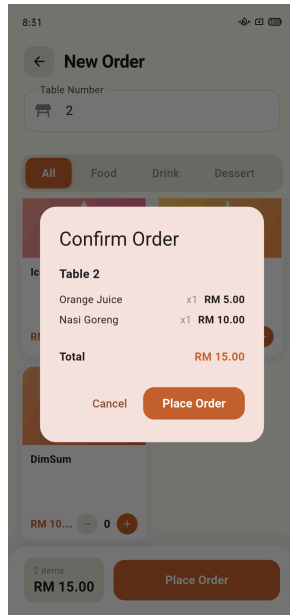
New Order Screen (Scrolled Down)

This screen shows the lower portion of the new order screen, displaying Ice Cream, French Fries, and DimSum with their quantity controls all set to zero. The total at the bottom still reflects the two previously selected items totalling RM 15.00.



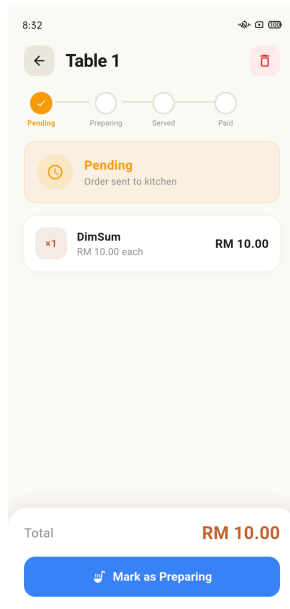
Confirm Order Dialog

This screen shows the order confirmation dialog that appears when the waiter taps Place Order, summarising the table number, each selected item with its quantity and price, and the total of RM 15.00. The waiter taps Place Order in the dialog to confirm and write the order atomically to Firestore.



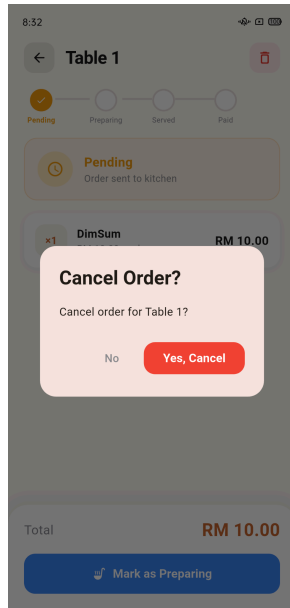
Order Detail Screen (Pending Status)

This screen shows the order detail for Table 1 with the status stepper at the top showing the order is currently at the Pending step. The status banner confirms the order has been sent to the kitchen and the Mark as Preparing button is shown at the bottom to advance the order to the next stage.



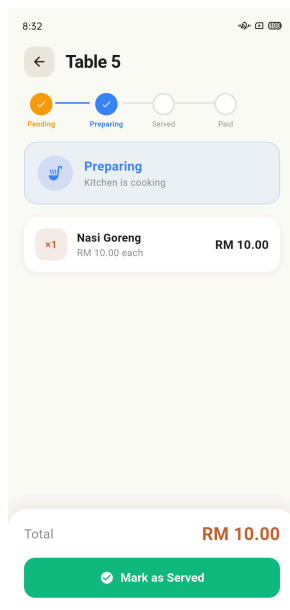
Cancel Order Confirmation Dialog

This screen shows the cancel order confirmation dialog that appears when the waiter taps the delete icon on a Pending order, asking whether to cancel the order for Table 1. Tapping Yes, Cancel will perform a batch delete of the order and all its associated order items from Firestore.



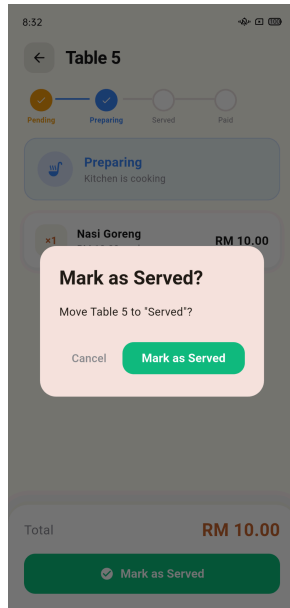
Order Detail Screen (Preparing Status)

This screen shows the order detail for Table 5 with the stepper advanced to the Preparing step, highlighted in blue. The status banner reads Preparing and shows the kitchen is currently cooking, with the Mark as Served button available at the bottom to advance the order forward.



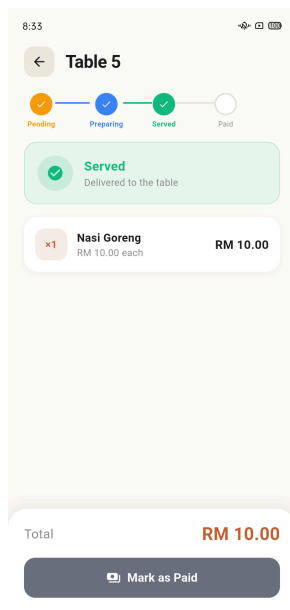
Mark as Served Confirmation Dialog

This screen shows the confirmation dialog asking the waiter to confirm moving Table 5 to the Served status. The waiter taps Mark as Served in the dialog to confirm the change, which updates the order in Firestore and returns to the Served filter in the orders list.



Order Detail Screen (Served Status)

This screen shows the order detail for Table 5 with the stepper now showing three completed steps up to Served, highlighted in green. The status banner confirms the order has been delivered to the table and the Mark as Paid button is shown at the bottom to complete the transaction.



Mark as Paid Confirmation Dialog

This screen shows the confirmation dialog asking the waiter to confirm moving Table 5 to the Paid status. Tapping Mark as Paid finalises the order, updates its status in Firestore, and navigates back to the Paid filter in the orders list.

